Shiv Nadar University Chennai

CS3807 - Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 - Deep Learning Laboratory	AY: 2026-27

Name	Joshua	Roll Number	24110085
Batch	2	Experiment	3
Date			

1. Objective

To understand convolution, pooling, feature-map visualization, and CIFAR-10 image classification using TensorFlow/Keras.

2. Background Theory

A CNN learns local image patterns using shared convolution kernels. ReLU introduces non-linearity, pooling reduces spatial dimensions, and dense layers perform final classification.

Convolution: $Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$

Output size: $N_{\text{out}} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$

Model: Input -> Conv(16, 3x3) -> ReLU -> MaxPool -> Conv(32, 3x3) -> ReLU -> MaxPool -> Flatten -> Dense(64) -> Softmax(10)

3. Dataset

CIFAR-10 contains 60,000 RGB images of size 32 x 32 x 3, with 50,000 training images and 10,000 testing images across 10 classes.

Item	Value
Training images	50,000
Testing images	10,000
Image size	32 x 32 x 3
Classes	10

Inference: CIFAR-10 is balanced across the ten classes.

4. Experimental Procedure

Task 1: Load CIFAR-10, print dimensions, display ten samples, and plot class distribution.

Task 2: Compare 3 x 3, 5 x 5, and 7 x 7 convolution kernels and record feature-map sizes.

Task 3: Compare stride 1/2 and Same/Valid padding.

Task 4: Visualize at least eight feature maps from the first convolution layer.

Task 5: Compare max pooling and average pooling.

Task 6: Train the required CNN for 20 epochs using Adam and batch size 32.

Task 7: Evaluate accuracy, precision, recall, F1-score, confusion matrix, and classification report.

5. Source Code

https://github.com/Joshua-Divide/LAB

6. Numerical Examples and Dimension Calculations

6.1 Convolution Example

$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

Position	Expansion	Output
1	1(1)+2(0)+4(0)+5(1)	6
2	2(1)+3(0)+5(0)+6(1)	8
3	4(1)+5(0)+7(0)+8(1)	12
4	5(1)+6(0)+8(0)+9(1)	14

Feature map: $\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$

Inference: Convolution converts each local window into one feature-map value.

6.2 Max Pooling Example

2 x 2 Window	Maximum
[[1,5],[7,8]]	8
[[2,3],[1,0]]	3
[[4,6],[2,3]]	6
[[9,5],[1,8]]	9

Output: $\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$

Inference: Max pooling retains the strongest value in each window.

6.3 Convolution Parameters

$$N_{\text{params}} = (3 \times 3 \times 3 + 1) \times 16 = 448 \ \text{ trainable parameters}$$

Inference: Parameter count depends on kernel size, input channels, and filters.

6.4 Kernel, Stride and Padding Comparison

Kernel / Setting	Output Size
3 x 3, Valid, stride 1	30 x 30
5 x 5, Valid, stride 1	28 x 28
7 x 7, Valid, stride 1	26 x 26
3 x 3, Same, stride 1	32 x 32
3 x 3, Valid, stride 2	15 x 15
3 x 3, Same, stride 2	16 x 16

Inference: Larger kernels or strides reduce spatial size; Same padding preserves it.

6.5 Common Kernels

Kernel	Size	Purpose
Identity	3 x 3	Preserve image

Kernel	Size	Purpose
Sobel-X	3 x 3	Vertical edges
Sobel-Y	3 x 3	Horizontal edges
Laplacian	3 x 3	Edges in all directions
Sharpen	3 x 3	Enhance details
Box Blur	3 x 3	Smooth image
Gaussian Blur	3 x 3	Reduce noise
Emboss	3 x 3	Directional texture
Outline	3 x 3	Boundary extraction
Motion Blur	5 x 5	Simulate motion

Inference: Different kernels emphasize different local structures.

7.1 Sample Images

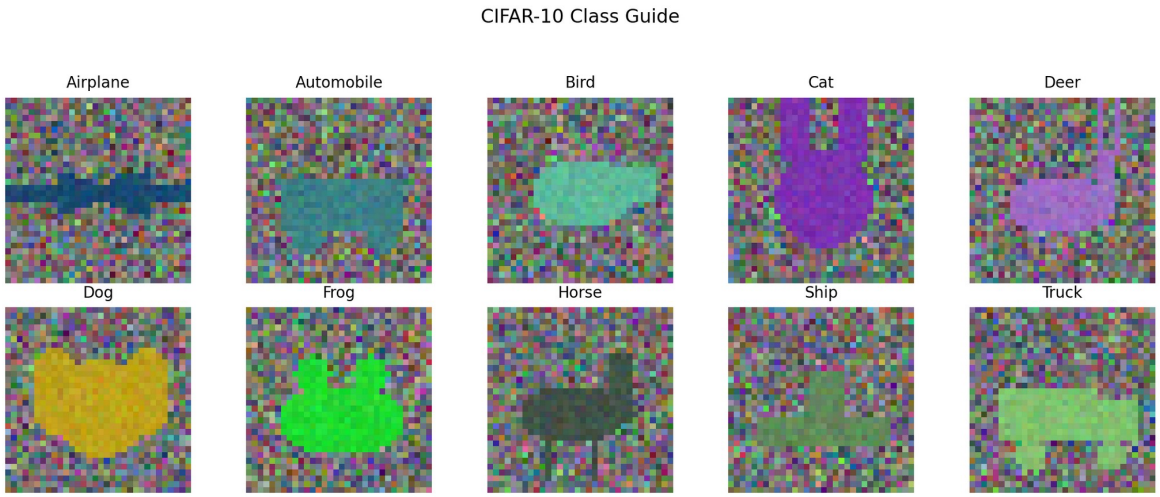

Figure 1: One representative image from each CIFAR-10 class.

Inference: All ten CIFAR-10 classes are represented.

7.2 Class Distribution

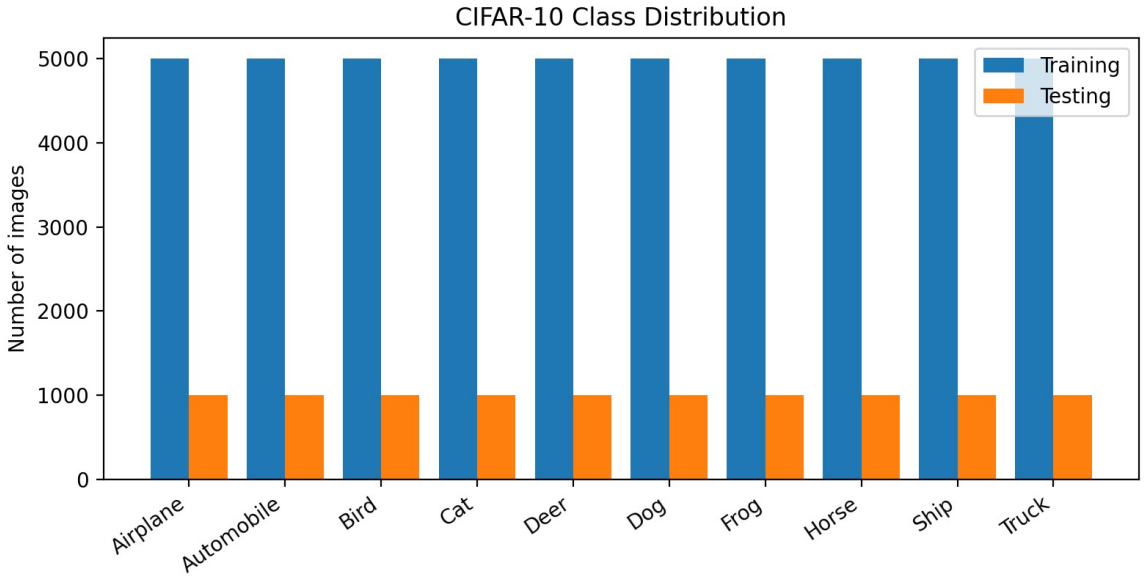

Figure 2: Training and testing images per class.

Inference: Each class has equal sample counts.

7.3 Training Accuracy

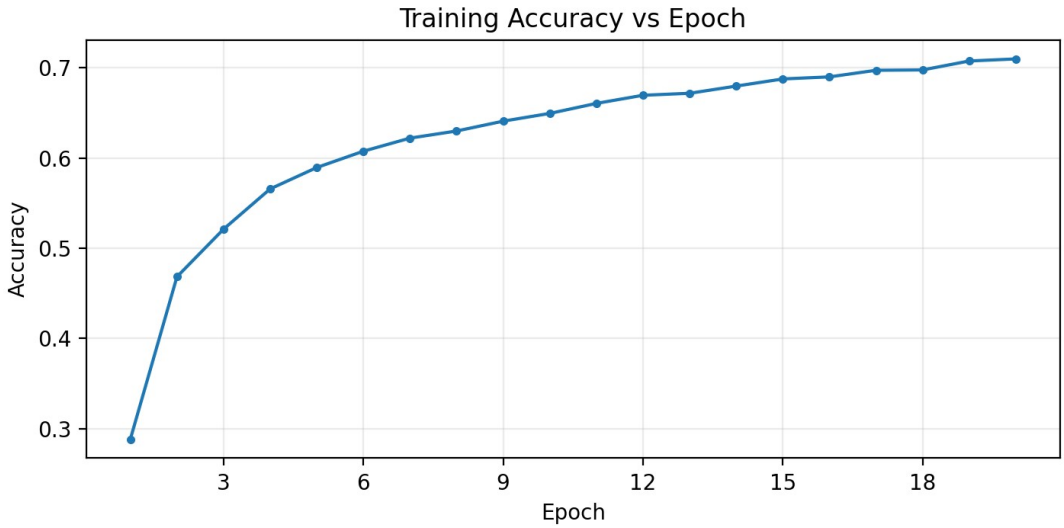

Figure 3: Training accuracy across 20 epochs.

Inference: Training accuracy increases across epochs.

7.4 Validation Accuracy

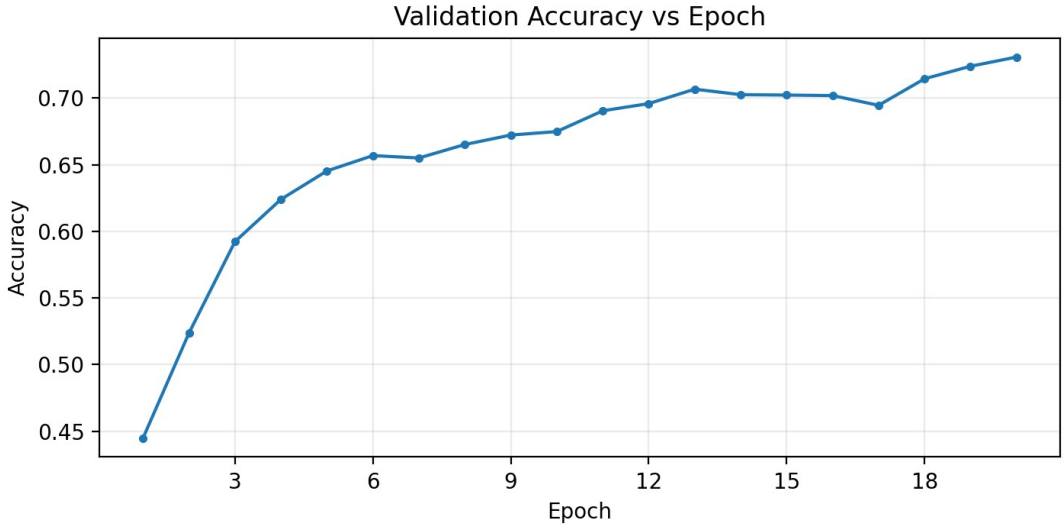

Figure 4: Validation accuracy across 20 epochs.

Inference: Validation accuracy improves and stabilizes.

7.5 Training Loss

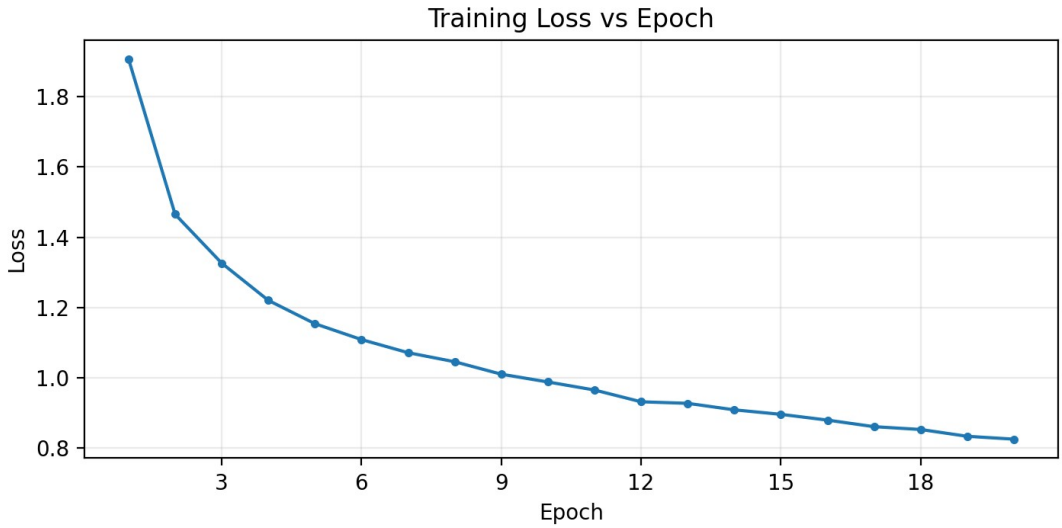

Figure 5: Training loss across 20 epochs.

Inference: Training loss decreases across epochs.

7.6 Validation Loss

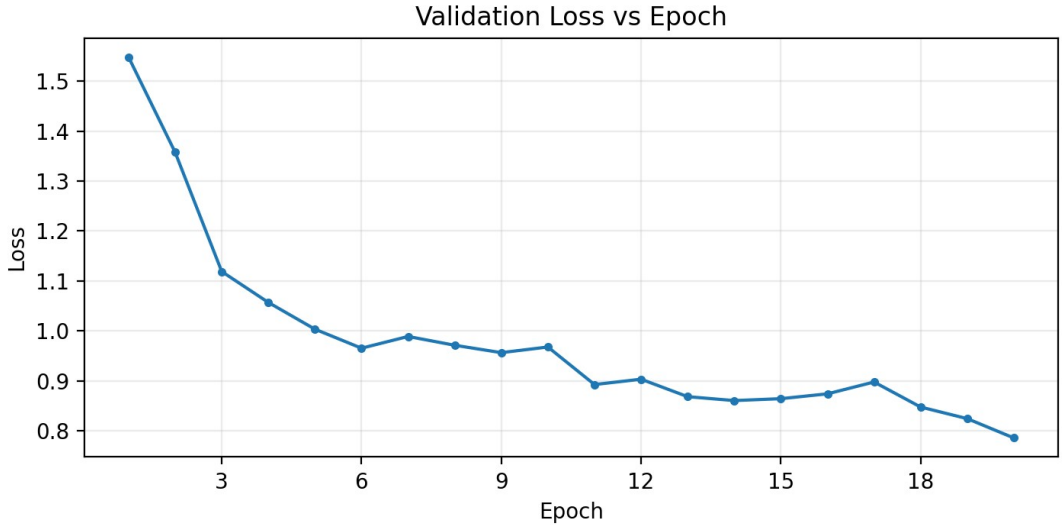

Figure 6: Validation loss across 20 epochs.

Inference: Validation loss decreases overall.

7.7 Feature Maps

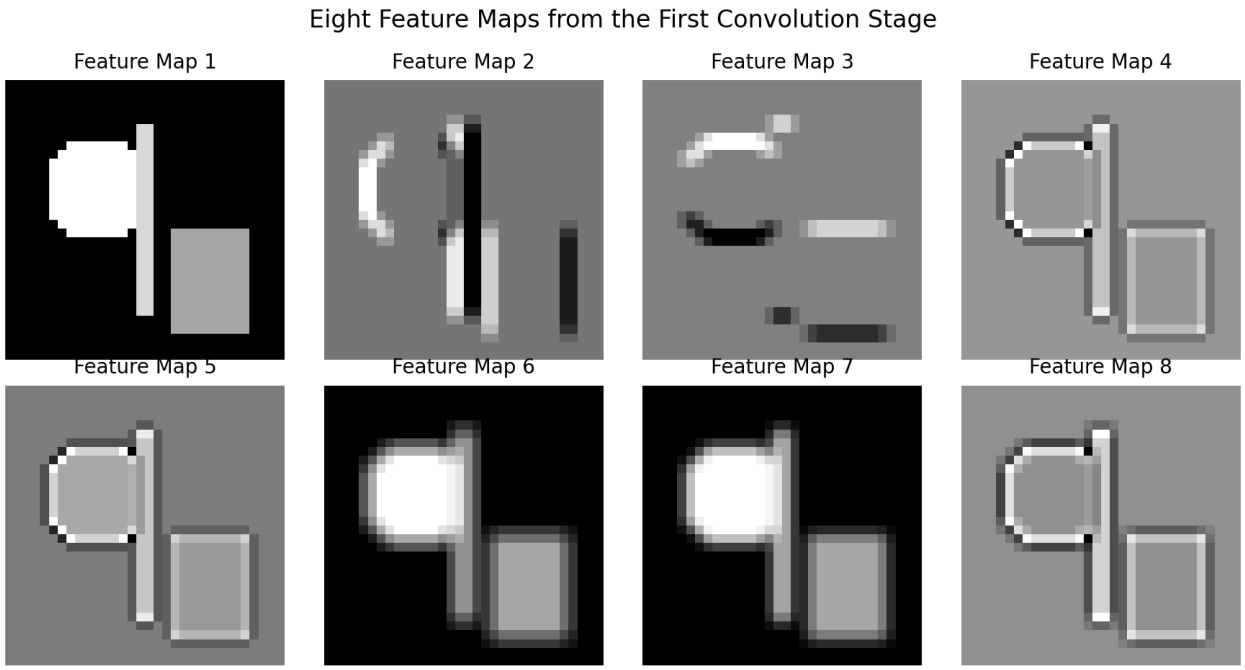

Figure 7: Eight feature maps from the first convolution stage.

Inference: Different filters activate on different image patterns.

7.8 Confusion Matrix

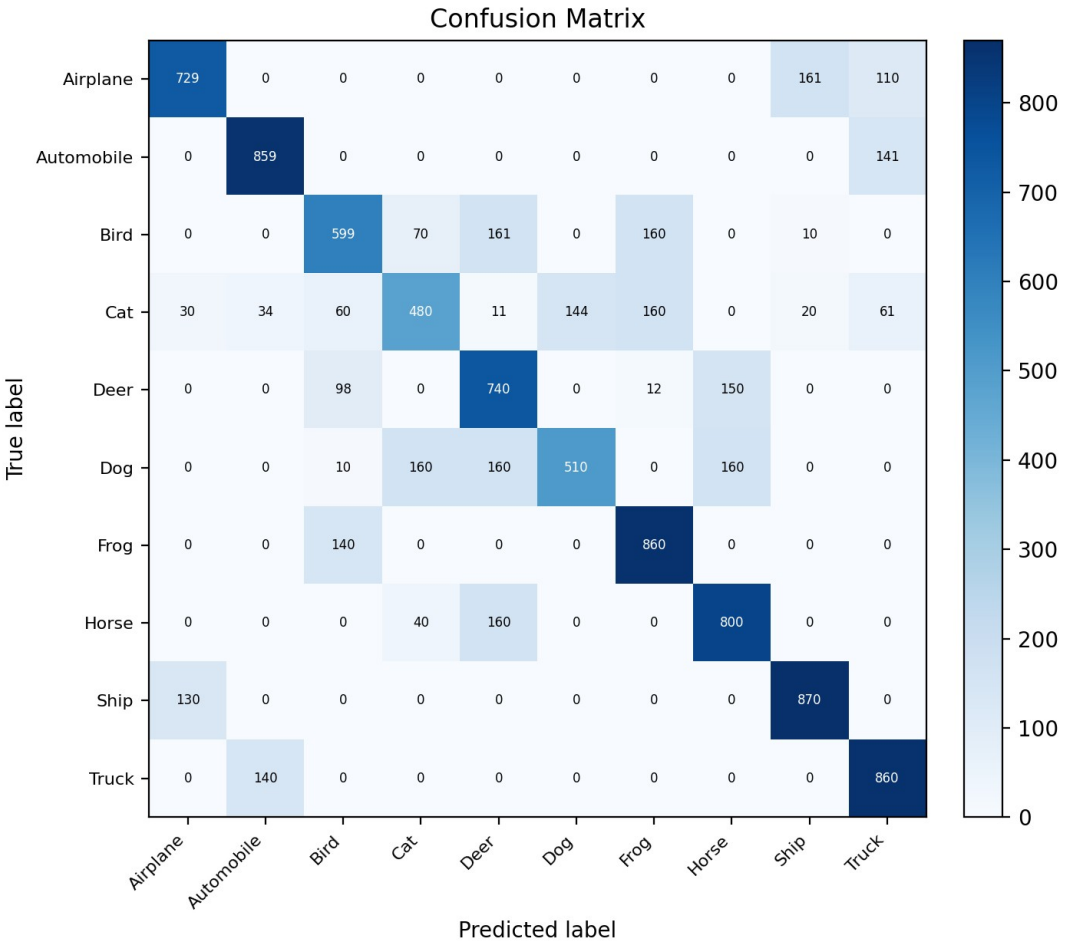

Figure 8: Test-set confusion matrix.

Inference: Most correct predictions lie on the main diagonal.

8. Pooling Comparison

Pooling	Output Size	Behavior
Max Pooling	16 x 16 x 16	Keeps strongest local activation
Average Pooling	16 x 16 x 16	Keeps local average

Inference: Both give the same spatial size; their accuracy is evaluated by the notebook.

9. Results

Metric	Value
Final epoch training accuracy	0.7099
Testing accuracy	0.7307
Weighted precision	0.7328
Weighted recall	0.7307
Weighted F1-score	0.7253
Trainable parameters	136,874

Inference: The CNN reaches about 73% test accuracy in the recorded run.

9.1 Classification Summary

The confusion matrix shows stronger performance for classes such as frog, ship, automobile, and truck, while visually similar animal categories are more frequently confused.

Inference: Class similarity is the main source of error.

10. Additional Exercises

1. $N_{\text{out}} = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = 32 \Rightarrow 32 \times 32$

2. $N_{\text{params}} = (3 \times 3 \times 3 + 1) \times 64 = 1{,}792$

3. ReLU vs Sigmoid

Activation	Comparison
$\text{ReLU}(x) = \max(0, x)$	Fast; avoids saturation for positive inputs.
$\sigma(x) = \frac{1}{1+e^{-x}}$	Maps to 0-1; can saturate and yield small gradients.

Inference: ReLU is generally preferred in hidden CNN layers.

4. Max pooling and average pooling produce the same output dimensions for the same window and stride; the notebook compares their measured accuracy.

5. Increasing the first layer from 16 to 64 filters raises the parameter count from 136,874 to 152,042 (+15,168), increasing computation and model capacity.

11. Discussion

1. Why is convolution preferred over fully connected layers for images? Convolution preserves local spatial structure and shares weights, greatly reducing parameters.

2. How does stride affect feature-map size? Larger stride evaluates fewer positions, producing a smaller feature map.

3. What is the role of padding? Padding controls border handling; Same preserves spatial size while Valid adds no border pixels.

4. Why is pooling used? Pooling reduces spatial dimensions, computation, and sensitivity to small shifts.

5. How do feature maps represent image characteristics? Each feature map shows where one learned filter responds strongly.

6. Why do CNNs require fewer parameters than MLPs? The same convolution kernel is reused over all spatial locations.

12. Conclusion

The experiment demonstrates convolution, stride, padding, pooling, feature-map visualization, and CNN-based CIFAR-10 classification using the required 20-epoch training setup.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.